Incident 1: Too Much Access

OUT1015 Self-Paced Lab: Activity Guide

Table of Contents

Incident 1: Too Much Access....1
Get Started....3
Overview....3
Meet the Team....3
What You Will Do....4
Prerequisites....4
Assumptions....5
Practice 1-1: Build the Starting State....6
Practice 1-2: Grant the Access Asha's Role Needs....8
Practice 1-3: Can Asha stop the instance?....10
Practice 1-4: Find the Grant That Allowed the Stop....11
Practice 1-5: Add an Explicit Deny and Test Again....13
Practice 1-6: Control Application Access with Single Sign-On....16
Practice 1-7: Review the Evidence and Close the Incident....20
Practice 1-8: Clean Up Your Tenancy....22
Troubleshooting....23
Setup....23
Policies and Permissions....23
Single Sign-On....23
Summit Retail Test Portal Code....25

Copyright © 2026, Oracle and/or its affiliates.

Get Started

Overview

Zero Trust starts from a simple position: no person, device, or workload is trusted because of where it sits or what it was allowed to do yesterday. Every request is checked when it is made. In Oracle Cloud Infrastructure (OCI), that check has two halves. An identity domain confirms who is asking, and Identity and Access Management (IAM) policies decide whether that identity may perform the requested action on the requested resource.

Every incident in this workshop happens at Summit Retail, a fictional online retailer that runs on OCI. In Incident 1, Asha, a member of the development team, is investigating an application issue. Her job requires her to view a production instance. During troubleshooting she sends a stop request, and it succeeds. You find out why, block the power action with an explicit deny policy, and then use an identity domain to control who can sign in to an application.

During the session, your instructor demonstrates this incident. This guide lets you build and run the same incident in your own tenancy afterward. You play two people: the administrator, who builds the scene and changes policies, and Asha, an ordinary user who tests what those policies allow.

The guide has eight practices. Practice 1-1 builds the starting state with a setup script in Cloud Shell, Practices 1-2 to 1-7 run the incident, and Practice 1-8 removes everything with one command. Allow about 90 minutes for a first run.

Meet the Team

The same people and resources appear in every practice.

Name	Who or what it is
Asha	Member of the development team. An ordinary user, not an administrator. Her role requires her to view instances.
Administrator	You. A member of the Administrators group in the Default identity domain.
SummitDevTeam	Asha's team group in the Default identity domain.
SummitTemporarySupport	A group that Asha joined for an old support task. Nobody removed her when the task ended.
SummitDemo	Compartment that holds the incident resources. Its parent compartment is aiw-iam-lab.

Copyright © 2026, Oracle and/or its affiliates.

Name	Who or what it is
prod-app-01	Small instance that stands in for a production server. It has no public IP address and serves no traffic.
Summit Retail Portal	A small test web page that you run on your own computer. It signs users in through Oracle single sign-on (SSO).

What You Will Do

- Build the starting state: two compartments, two groups, the user Asha, a small instance, and one forgotten policy.
- Separate authentication (who Asha is) from authorization (what Asha may do) by using one real request.
- Add a read policy for SummitDevTeam, then watch Asha stop the instance anyway.
- Find the existing group membership and policy that allowed the stop.
- Add an explicit deny for instance power actions and confirm that Asha can still view the instance but can no longer stop it.
- Register an application in an identity domain and control who can sign in to it by changing a group.
- Record the evidence for each test, close the incident, and clean up your tenancy.

Prerequisites

- An OCI tenancy where you are a member of the Administrators group in the Default identity domain. A trial or personal tenancy works. Do not use a shared production tenancy.
- IAM deny policies enabled in the tenancy, or permission to enable them. Enabling deny policies is permanent and cannot be undone. Read Practice 1-5 before you decide.
- Service limits for one small compute instance, a virtual cloud network (VCN), and a subnet. An Always Free eligible shape, such as VM.Standard.A1.Flex with 1 OCPU, is enough.
- An email address you can read, for Asha's account. Many mail services accept an alias such as `you+asha@example.com`.
- For Practice 1-6, Python 3.8 or later and OpenSSL on your computer, to run the Summit Retail test portal that comes with this guide. macOS and Linux include OpenSSL; on Windows, it comes with Git for Windows.
- Two browsers, or one browser with private (incognito) windows, so that the administrator and Asha are never signed in to the same window.

Copyright © 2026, Oracle and/or its affiliates.

Assumptions

- Three browser windows stay open throughout. Window A is your OCI Console session as the administrator. Window B is a private window signed in to the OCI Console as Asha. Window C is a private window for the application in Practice 1-6; you open a fresh one for every sign-in test. Label the windows so that you always know who is acting.
- You work in your tenancy's home region. IAM changes are made in the home region, and keeping the instance there keeps every step in one place.
- Asha is an ordinary user. Members of the Administrators group in the Default identity domain are exempt from deny policies, so a deny test performed as the administrator proves nothing.
- Text in a light box with a red left edge is text that you enter in the OCI Console. A dark box is a command for a terminal. A gray box shows expected output or reference information.
- IAM changes take a short time to take effect. Wait, then refresh the page before you judge a result. Never judge a result from a page that you loaded before the change.
- Console labels move between releases. If a button is not where this guide says, look for the same action under the **Actions** menu or the **More actions** icon.
- Each expected result is marked Expected. Record what you actually see in the evidence worksheet in Practice 1-7, which is not always what the guide predicts.

Note:

- Text in angle brackets, such as `<home-region>`, is a placeholder. Replace it with your own value and do not type the angle brackets.
- The resources in this guide can incur charges outside the Always Free limits. Practice 1-8 removes them.
- Use names exactly as written. Policy statements refer to groups and compartments by name, so a different spelling breaks the test.

Copyright © 2026, Oracle and/or its affiliates.

Practice 1-1: Build the Starting State

Acts: you, as the administrator in Cloud Shell and Window A, and as Asha in Window B · About 10 minutes

In the session, the starting state was ready before the demonstration began. Here a setup script builds it for you in OCI Cloud Shell, which signs in as you, so you need no API key. One part of the scene needs care: the script puts Asha in two groups and gives the second group a policy. In the story, that membership and policy are left over from an old support task, and nobody remembers them. When you reach Practice 1-3, set aside what you know from this list and reason only from the policy that you add in Practice 1-2.

The script creates	Details
Compartment `aiw-iam-lab`	In your tenancy root, with the child compartment `SummitDemo`
User `summit-asha` (Asha)	In the Default identity domain, with the email address that you give
Groups `SummitDevTeam` and `SummitTemporarySupport`	Asha belongs to both
Policy `Summit-SummitTemporarySupport-Use`	Attached to `aiw-iam-lab`: the forgotten support grant
Instance `prod-app-01`	A small Always Free VM in `SummitDemo`, with no public IP address and no internet route

Run the Setup Script

1. In Window A, sign in to the OCI Console as the administrator and select your home region. In the Console header, click the **Developer tools** icon and select **Cloud Shell**.

Copyright © 2026, Oracle and/or its affiliates.

2. Download the package, and build the starting state. Replace the email with an address you can read; Asha's account notices go there:

```
$ REPO=https://github.com/SaurabhOCI/aiworld-2026-zero-trust-labs
$ ZIP=aiw-iam-sso-lab.zip
$ wget -q -O ~/$ZIP $REPO/raw/main/self-paced/$ZIP
$ unzip -o -q ~/$ZIP -d ~/aiw-iam-lab && cd ~/aiw-iam-lab
$ python3 lab.py build --asha-email you+asha@example.com
```

```
Creating compartment aiw-iam-lab
...
Launching prod-app-01 (VM.Standard.A1.Flex) in ...
prod-app-01: RUNNING
Build complete.
  OK   asha_active
  ...
IAM lab: STARTING STATE
```

3. Show Asha's one-time password:

```
$ python3 lab.py asha-password
```

```
Asha signs in as: summit-asha   one-time password: ...
```

Prepare Asha's Window

4. Open a private window, which becomes Window B. Sign in to the OCI Console as `summit-asha` with the one-time password, and choose a new password when asked. Select your home region.

5. In Window B, select **Compute** > **Instances**, select the **SummitDemo** compartment under **aiw-iam-lab**, and open **prod-app-01**. Leave the window on this page.

Note:

- The policy is attached to aiw-iam-lab, so `compartment SummitDemo` resolves to its direct child. If you attach a policy to the root compartment instead, write the path `aiw-iam-lab:SummitDemo`.
- The group name in a policy carries its identity domain as a prefix: `Default/SummitTemporarySupport`.
- If the build stops with a message, fix what it says and run the same command again. It continues where it stopped.
- Asha's Console session may show authorization messages in panels that have nothing to do with the instance, such as monitoring or networking. Her access is narrow, so this is expected.

Copyright © 2026, Oracle and/or its affiliates.

Practice 1-2: Grant the Access Asha's Role Needs

Acts: you, as the administrator in Window A · About 10 minutes

Every request that Asha makes passes two separate decisions:

Decision	Question it answers	Where OCI makes it
Authentication	Who is Asha?	The identity domain, at sign-in, using her password and any multifactor authentication (MFA).
Authorization	May Asha do this, to this resource, here?	OCI IAM, on every API request, using her group memberships and the policies that apply.

Default is an identity domain: it holds Asha and her groups. SummitDemo is a compartment: it holds prod-app-01. A policy connects the two by naming a group, an action, and a compartment. Asha needs to look at prod-app-01 while she investigates, so least privilege means giving her team permission to read instances in SummitDemo, and nothing more.

Tasks

1. Answer this before you continue: if Asha completes MFA, should she automatically be able to stop prod-app-01?

2. In Window A, select **Identity & Security** > **Policies**. From the **Compartment** list, select **aiw-iam-lab**.

3. Click **Create Policy** and enter the following values:

Field	Value
Name	`Summit-SummitDevTeam-Read`
Description	`Incident 1: SummitDevTeam can view instances in SummitDemo`
Compartment	`aiw-iam-lab`

4. In **Policy Builder**, select **Show manual editor** and enter this statement:

```
allow group Default/SummitDevTeam to read instances
in compartment SummitDemo
```

5. Click **Create**. The policy details page opens and shows the statement.

Copyright © 2026, Oracle and/or its affiliates.

6. Read the statement in three parts. Who: members of the SummitDevTeam group in the Default identity domain. What: read instances, which means viewing instance details. Where: the SummitDemo compartment.

7. Considering only this policy, write down two predictions in row 1 of the evidence worksheet in Practice 1-7: can Asha view prod-app-01, and can she stop it?

Note:

- MFA makes the sign-in stronger, but a successful sign-in grants no permission on any resource. Stopping prod-app-01 still needs a matching policy.
- An identity domain and a compartment are different things. The domain holds identities. The compartment holds cloud resources. Neither one grants access by itself.
- The `read instances` permission lets a user view instance details. It contains no power actions such as start or stop.

Copyright © 2026, Oracle and/or its affiliates.

Practice 1-3: Can Asha stop the instance?

Acts: you, as Asha in Window B and as the administrator in Window A · About 5 minutes

The policy you just wrote grants SummitDevTeam read access. Now Asha tries the action that started the incident.

Tasks

1. Switch to Window B. Refresh the prod-app-01 details page and confirm the instance name, the compartment SummitDemo, and the state **Running**. Record whether Asha can view the instance.

2. Click **Stop**. In the dialog box, leave the option to force the stop deselected and confirm the stop.

3. Watch the state change to **Stopping** and then to **Stopped**.

4. Switch to Window A and refresh the same instance. Confirm that the administrator also sees **Stopped**, so you know that both windows show the same instance.

5. Record the actual result in row 1 of the worksheet.

```
Expected: Asha can view prod-app-01, and the stop succeeds.
The instance moves through Stopping to Stopped.
```

6. Ask yourself: the policy you wrote allows only read. Why did a stop succeed?

Note:

- A read permission cannot authorize a stop. Something else that applies to Asha must allow it.
- Adding an allow policy never removes access. Allow statements only add permissions, so the new read policy did not narrow anything that Asha could already do.
- If a stop attempt fails, read the message before drawing a conclusion. A network error, or a state error on an instance that is already stopping, says nothing about permissions.

Copyright © 2026, Oracle and/or its affiliates.

Practice 1-4: Find the Grant That Allowed the Stop

Acts: you, as the administrator in Window A · About 10 minutes

The stop succeeded, so some policy allowed it. Investigate the way an administrator would on a real incident: look at Asha's complete group list, then at every policy that applies to SummitDemo. Change nothing while you search.

Tasks

1. In Window A, select **Identity & Security** > **Domains**, click the **Default** domain, and click the **User management** tab. In the **Users** section, click **summit-asha**.

2. Click the **Groups** tab. Asha belongs to two groups, not one:

```
SummitDevTeam
SummitTemporarySupport
```

3. Select **Identity & Security** > **Policies**. From the **Compartment** list, select **aiw-iam-lab**. Two policies are attached: `Summit-SummitDevTeam-Read` and `Summit-SummitTemporarySupport-Use`.

4. Click **Summit-SummitTemporarySupport-Use** and read its statement:

```
allow group Default/SummitTemporarySupport to use instances
in compartment SummitDemo
```

5. Answer this: does the new read policy cancel this use policy?

6. Return the instance to its starting point for the next test. In Window A, open prod-app-01. When its state is **Stopped**, click **Start** and wait until the state is **Running**. Keep both groups and both allow policies in place.

Note:

- No. Matching allow grants combine. Asha receives the permissions of every group she belongs to, so read from SummitDevTeam plus use from SummitTemporarySupport gives her use.
- The `use instances` permission includes the power actions start, stop, and reboot. It also includes other permissions, such as updating an instance and attaching volumes.
- This practice explains the stop that you already saw. No membership was added and no policy was created.

Copyright © 2026, Oracle and/or its affiliates.

- On a real incident, also check policies attached to the root compartment, which can reach SummitDemo too.
- Membership that was granted for a finished task and never removed is exactly the kind of access that Zero Trust asks you to find and take away.

Copyright © 2026, Oracle and/or its affiliates.

Practice 1-5: Add an Explicit Deny and Test Again

Acts: you, as the administrator in Window A and as Asha in Window B · About 15 minutes

Removing SummitTemporarySupport is the real fix, and you do it in Practice 1-7. First, add a guardrail that holds even when an extra allow is left behind: an explicit deny. OCI evaluates a request in this order:

```
1  Does a deny statement match?    Yes: denied (explicit deny)
2  Does an allow statement match?  Yes: allowed
3  Neither matches                 Denied (implicit deny)
```

Check That Deny Policies Are Enabled

1. In Window A, select **Identity & Security** > **Policies**. Open the **Actions** menu and select **Policy settings**.

2. If deny policies are already enabled, close the settings and continue with the next section. If they are not enabled, decide now whether to enable them in this tenancy. Enabling is permanent: once enabled, the feature cannot be turned off.

3. To enable them, select **Enable IAM Deny Policy** and confirm with **Enable**. When OCI offers to create a default deny policy that stops other users from writing deny statements, select **Yes**.

Add the Deny and Test

4. Confirm that prod-app-01 is **Running**.

5. In **Policies**, from the **Compartment** list, select **aiw-iam-lab** and click **Create Policy**. Enter the following values:

Field	Value
Name	`Summit-Deny-Power`
Description	`Incident 1: block instance power actions for SummitDevTeam in SummitDemo`
Compartment	`aiw-iam-lab`

6. In **Policy Builder**, select **Show manual editor** and enter this statement exactly, including the plural ACTIONS:

```
deny group Default/SummitDevTeam to {INSTANCE_POWER_ACTIONS}
in compartment SummitDemo
```

Copyright © 2026, Oracle and/or its affiliates.

7. Click **Create**.

8. Write down your predictions in row 3 of the worksheet. Both allow policies are still in place. Can Asha view prod-app-01? Can she stop it?

9. Wait for the change to take effect. In Window B, refresh the prod-app-01 details page. Confirm that Asha can still see the instance details and that the state is **Running**.

10. Click **Stop** again and confirm. Record exactly what the Console does: an error message and its wording, or a **Stop** control that is disabled or missing.

11. In Window A, refresh prod-app-01 and confirm that it is still **Running**.

```
Expected: Asha can still view prod-app-01.
The stop is blocked, and the instance stays Running.
```

12. (Optional) To see the refusal as an API response, add an API key for Asha, configure an OCI CLI profile with it, and send the same stop request from a terminal:

```
$ oci compute instance action --action STOP \
  --instance-id <prod-app-01-ocid> \
  --profile <asha-profile> --region <home-region>
```

```
Expected: the request is refused with an authorization error,
such as NotAuthorizedOrNotFound, and the instance stays Running.
```

13. Ask yourself what changed. Asha is the same user and prod-app-01 is the same instance. Only the permission decision changed.

Note:

- Use `INSTANCE_POWER_ACTIONS` exactly. It is the permission behind the OCI start, stop, and reboot requests.
- The deny removes only that permission. It does not remove the other permissions in the use grant, and it does not stop someone with operating system access from shutting the server down from inside, for example through SSH.
- The statement applies only to SummitDevTeam and only in SummitDemo. Other groups and other compartments are not affected.
- Only members of the Administrators group in the Default domain can enable deny policies. The default deny policy that OCI offers keeps everyone else from writing deny statements.

Copyright © 2026, Oracle and/or its affiliates.

- A `NotAuthorizedOrNotFound` response also appears when the profile, region, or OCID is wrong. Check all three before you treat it as evidence of the deny.

Copyright © 2026, Oracle and/or its affiliates.

Practice 1-6: Control Application Access with Single Sign-On

Acts: you, as the administrator in Window A and as Asha in Window C · About 30 minutes

You have controlled what Asha can do to an OCI instance. Summit Retail also has a growing support team that needs a support portal. With a local account in every application, someone has to create each account, reset forgotten passwords, and remove access when the work ends. In this practice you register the portal in an identity domain, require an assignment before anyone can use it, and grant access by changing a group.

The portal is a small test program, `summit_portal.py`, that runs on your own computer. It uses only the Python standard library and signs users in with OpenID Connect (OIDC). Get it from the workshop files, or copy it from Summit Retail Test Portal Code at the end of this guide. It is for testing only and keeps sessions in memory.

Identity domains expect application addresses to use HTTPS. The portal runs on your own computer, so you give it a self-signed certificate for `localhost`. The identity domain never connects to the portal: it only sends your browser back to the redirect address. A certificate that only your own browser sees is enough.

Prepare the Portal Folder

1. Create a new folder, for example `summit-portal`, and save `summit_portal.py` in it.

2. Open a terminal in that folder and create a test certificate for `localhost`. On Windows, run the command in Git Bash:

   ```
   $ openssl req -x509 -newkey rsa:2048 -nodes -days 7 \
     -keyout key.pem -out cert.pem -subj "/CN=localhost"
   ```

3. Confirm that the folder now holds three files: `summit_portal.py`, `cert.pem`, and `key.pem`. When the portal finds the two certificate files, it serves HTTPS at `https://localhost:8443`.

Register the Portal in an Identity Domain

4. In Window A, select **Identity & Security** > **Domains** and click the identity domain to use. The Default domain works, and Asha already exists there. A separate domain, such as the Test-Domain used in the demonstration, keeps application users apart from OCI administrators; if you use one, create Asha there as you did in Practice 1-1.

Copyright © 2026, Oracle and/or its affiliates.

5. On the domain details page, copy the **Domain URL** and keep it for later. It looks like `https://idcs-<id>.identity.oraclecloud.com:443`.

6. Click the **User management** tab. In the **Groups** section, create a group named `SummitPortalUsers` with no members.

7. Click **Integrated applications**, click **Add application**, select **Confidential Application**, and click **Launch workflow**.

8. Enter the following values, then click **Next**:

Field	Value
Name	`Summit Retail Portal`
Application URL	`https://localhost:8443`
Enforce grants as authorization	Selected

9. Select **Configure this application as a client now** and enter the following values:

Field	Value
Grant types	**Authorization code**
Redirect URL	`https://localhost:8443/callback`

10. Click **Next**, then click **Finish**.

11. In the **Application added** dialog box, record the **Client ID** and **Client secret** in a safe place, and click **Close**. Treat the client secret like a password.

12. On the application details page, click **Activate** and confirm.

13. In the application's **Groups** resource, assign the group **SummitPortalUsers**.

Copyright © 2026, Oracle and/or its affiliates.

Start the Portal

14. In the terminal, in the portal folder, start the portal. On Windows, use `py` instead of `python3`:

```
$ python3 summit_portal.py
```

```
Summit Retail test portal

Identity domain URL (https://idcs-...): <domain-url>
Client ID: <client-id>
Client secret (hidden):

Portal running at https://localhost:8443
Redirect URL: https://localhost:8443/callback
```

15. Answer the three prompts with the domain URL, client ID, and client secret that you recorded. The secret is not shown as you type. Leave the terminal open; the portal runs until you press Ctrl+C, and it prints a line for every sign-in.

Test Access Before and After Assignment

16. Open a fresh private Window C and go to `https://localhost:8443`. The browser warns that the certificate is not trusted, because you signed it yourself. Open the advanced details and continue to `localhost`.

17. Click **Continue with Oracle SSO**. The browser moves to the Oracle sign-in page for your identity domain. Sign in as `summit-asha`. Record the result in row 4 of the worksheet.

```
Expected: Asha's password is accepted, but she does not
reach the portal. Either the Oracle page or the portal
reports that access is refused.
```

18. In Window A, open the domain's **User management** tab and click **summit-asha**. Click the **Groups** tab, click **Assign user to groups**, select **SummitPortalUsers**, and click **Assign user**.

19. Close every Window C. Open a new private Window C, go to `https://localhost:8443`, continue past the certificate warning, click **Continue with Oracle SSO**, and sign in as `summit-asha` again. Record the result in row 5 of the worksheet.

```
Expected: the portal shows Welcome, with Asha's name
and "Account managed by: Oracle SSO (no portal
password)". The terminal prints Signed in: summit-asha.
```

20. Ask yourself: Asha is now signed in to the portal through SSO. May she stop prod-app-01?

Copyright © 2026, Oracle and/or its affiliates.

21. (Optional) Remove Asha from SummitPortalUsers, close every Window C, open a new one, and sign in again. Record the result in row 6 of the worksheet.

Note:

- No. The identity domain confirms who Asha is and whether she is assigned to the portal. OCI IAM still decides every request on an OCI resource, based on the identity that sends it. The portal makes no OCI requests at all.
- Without **Enforce grants as authorization**, any user who can authenticate in the domain can use the portal. The setting is what makes group membership matter.
- You created no portal password. A group change in the Console granted access, and the same group can serve the whole support team.
- A membership change does not end a portal session that is already open. Always test in a brand-new private window, and allow time for the change to take effect.
- The self-signed certificate is only for this test on your own computer. A portal on a public server, like the one in the demonstration, needs a certificate from a public certificate authority, for example a free certificate from Let's Encrypt issued with the certbot tool.
- Without `cert.pem` and `key.pem`, the portal serves plain HTTP at `http://localhost:8000`. Oracle documents an **Allow HTTP URLs** option for testing, but HTTPS avoids any doubt about whether your domain accepts an HTTP redirect address.
- Identity domains also offer password recovery, sign-on policies that require MFA, sign-in reports, and SSO across several integrated applications.
- An incorrect password or an account activation error is an authentication problem. It is not evidence about application assignment.

Copyright © 2026, Oracle and/or its affiliates.

Practice 1-7: Review the Evidence and Close the Incident

Acts: you, as the administrator in Window A and as Asha in Window B · About 15 minutes

In this practice you compare what you predicted with what you saw, answer four policy questions, and then remove the access that caused the incident.

Tasks

1. Complete the evidence worksheet. Write your observation in your own words, separately from the expected result.

Row	Test	Expected	What you observed
1	Read policy added; Asha views, then stops prod-app-01	View allowed. Stop allowed.	
2	SummitTemporarySupport membership and policy found	No change. The finding explains row 1.	
3	Both allows plus the power deny; Asha views, then stops	View allowed. Stop blocked. Instance stays Running.	
4	Asha signs in to the application before assignment	Application access refused.	
5	Asha added to SummitPortalUsers; fresh sign-in	Application opens as Asha.	
6	(Optional) Asha removed from the group; fresh sign-in	Application access refused.	

2. Answer each question, and name the policy that decides the result:
   - Question 1: Another group gives SummitDevTeam members use instances in SummitDemo. The power deny still matches. Can Asha stop prod-app-01?
   - Question 2: Every allow is removed and only the deny remains. Can Asha still read the instance details?
   - Question 3: An instance sits in an unrelated compartment. Does this deny alone decide whether Asha can stop it?
   - Question 4: Which system authenticates the application user, and which decision controls the OCI power action?

Copyright © 2026, Oracle and/or its affiliates.

3. Close the incident. In Window A, open the **Default** domain, click the **User management** tab, click **summit-asha**, and click the **Groups** tab. Remove Asha from **SummitTemporarySupport**.

4. In **Identity & Security** > **Policies** for **aiw-iam-lab**, delete **Summit-SummitTemporarySupport-Use**. No one needs it now.

5. Keep **Summit-SummitDevTeam-Read**, which is the access that Asha's role needs. Keep **Summit-Deny-Power** only if Summit Retail decides that SummitDevTeam must never run power actions in SummitDemo.

6. Verify the final state in Window B. After the change takes effect, refresh prod-app-01: Asha can view the instance, and her stop attempt is refused.

Note:

- Question 1: No. An applicable explicit deny wins over any number of allows.
- Question 2: No. A deny grants nothing. Without an allow, the request falls through to the implicit deny.
- Question 3: No. Evaluate the policies that apply to that compartment. This deny is scoped to SummitDemo.
- Question 4: The identity domain authenticates the application user and enforces the application assignment. OCI IAM policies decide the power action, based on the identity that sends the OCI request.
- Once SummitTemporarySupport is gone, Asha's stop is refused even without the deny, because no allow matches. The deny remains useful as a rule that holds if someone grants extra access again by mistake.

Copyright © 2026, Oracle and/or its affiliates.

Practice 1-8: Clean Up Your Tenancy

Acts: you, as the administrator in Window A and Cloud Shell · About 15 minutes

Remove what you created so that nothing keeps running or keeps granting access.

Tasks

1. Sign out of the OCI Console in Window B and close every private window.

2. If you completed Practice 1-6, press Ctrl+C in the portal terminal and delete the portal folder, including `cert.pem` and `key.pem`. In the identity domain, click **Integrated applications**, deactivate **Summit Retail Portal**, and then delete it. Delete the group **SummitPortalUsers**.

3. In Cloud Shell, run the teardown. It deletes the instance and its network, every policy in aiw-iam-lab, the user summit-asha, both groups, and both compartments:

```
$ cd ~/aiw-iam-lab
$ python3 lab.py teardown --yes
```

```
Terminating prod-app-01
...
Deleted user summit-asha
...
Deleting compartment SummitDemo: OCI finishes this in the
background.
All lab resources are gone. To delete the empty compartment
aiw-iam-lab too, run this teardown again in about an hour. ...
```

Note:

- Deny policies stay enabled in your tenancy. That setting cannot be reversed.
- OCI can take 30 minutes or more to delete a compartment, and `aiw-iam-lab` can be deleted only after `SummitDemo`. Run the same teardown again later to delete it; the second run ends with `Teardown complete.` An empty compartment costs nothing.

Repeat the Incident Instead

4. To run the incident again from the start rather than cleaning up, run `python3 lab.py reset` in `~/aiw-iam-lab`. It deletes Summit-DevTeam-Read and Summit-Deny-Power, restores Asha's two groups and the support policy, and starts prod-app-01 if it is stopped.

Copyright © 2026, Oracle and/or its affiliates.

Troubleshooting

Setup

- Asha's activation email does not arrive: check spam, then select **summit-asha** in the user list and use **Reset password** to send a new link.
- The instance cannot be created: the shape is out of capacity or over your service limit. Try another availability domain or another small shape.
- Asha cannot see prod-app-01 at all: check that she is in SummitTemporarySupport, that Summit-SummitTemporarySupport-Use is attached to aiw-iam-lab, and that Window B is in your home region with SummitDemo selected.
- Creating a policy fails because the compartment cannot be found: the policy is attached in the wrong place. Attach it to aiw-iam-lab, or write the full path aiw-iam-lab:SummitDemo.

Policies and Permissions

- The stop is refused in Practice 1-3: check that Window B is signed in as Asha, not the administrator, and that Asha is in SummitTemporarySupport.
- Asha sees authorization messages in Console panels that have nothing to do with the instance: these are expected with narrow grants. They are not a failed test. Do not add broad grants to clear them.
- The deny statement is rejected: deny policies are not enabled in the tenancy, or the default deny policy prevents your account from writing deny statements. Sign in as a member of the Administrators group in the Default domain.
- The stop still succeeds after the deny: wait and refresh, then check the plural INSTANCE_POWER_ACTIONS, the group name Default/SummitDevTeam, and that Asha is not in the Administrators group.

Single Sign-On

- Any signed-in user can open the application: **Enforce grants as authorization** is not selected, or the application is not activated.
- Access is still refused after Asha joins SummitPortalUsers: an earlier session is being reused. Close every private window, wait briefly, and try again in a new one.
- The Oracle page reports an invalid redirect: the **Redirect URL** must match the address that the portal prints at startup exactly, including `https`, `localhost`, and the port `8443`.

Copyright © 2026, Oracle and/or its affiliates.

- The portal starts on http://localhost:8000 instead of https://localhost:8443: `cert.pem` and `key.pem` are not in the same folder as `summit_portal.py`.
- The openssl command is not found on Windows: open Git Bash, which comes with Git for Windows, and run the command there.
- The browser will not let you continue past the certificate warning: use a private window in Chrome, Edge, Firefox, or Safari, and open the advanced details on the warning page. The warning appears on every new private window.
- The Oracle page reports an invalid client, or the portal shows Sign-in failed: check the client ID and secret, and that the application is activated. Restart the portal to enter them again.
- The portal stops with CERTIFICATE_VERIFY_FAILED on macOS: Python cannot find trusted certificates. Run Install Certificates.command from your Python folder in Applications, then start the portal again.
- The portal stops because the address is already in use: another program is using port 8443. Close it, or start the portal with the environment variable `PORTAL_PORT=8444` and change both URLs in the application to port 8444.
- The Oracle sign-in page reports a password or activation problem: that is an authentication issue. Resolve it before you test application access.

Copyright © 2026, Oracle and/or its affiliates.

Summit Retail Test Portal Code

Save this code as `summit_portal.py`. It needs Python 3.8 or later and no extra packages. Use it only for testing on your own computer.

```
"""Summit Retail test portal for the OUT1015 Zero Trust workshop.

A throwaway web page that signs users in through an OCI identity domain
with OpenID Connect. It shows whether the domain lets a user in.
Standard library only; Python 3.8 or later. For testing only: it keeps
sessions in memory. If cert.pem and key.pem are in the same folder, it
serves HTTPS on port 8443; otherwise it serves plain HTTP on port 8000.
"""
import base64
import getpass
import hashlib
import html
import json
import os
import secrets
import ssl
import sys
import urllib.error
import urllib.parse
import urllib.request
from http.server import BaseHTTPRequestHandler, HTTPServer

HERE = os.path.dirname(os.path.abspath(__file__))
CERT = os.path.join(HERE, "cert.pem")
KEY = os.path.join(HERE, "key.pem")
TLS = os.path.exists(CERT) and os.path.exists(KEY)
PORT = int(os.environ.get("PORTAL_PORT", "8443" if TLS else "8000"))
BASE = f"{'https' if TLS else 'http'}://localhost:{PORT}"
REDIRECT_URI = f"{BASE}/callback"
PENDING = {}   # state -> nonce and PKCE verifier for a sign-in in progress
SESSIONS = {}  # cookie value -> signed-in user
COOKIE = "portal_session"
FLAGS = "Path=/; HttpOnly; SameSite=Lax" + ("; Secure" if TLS else "")

STYLE = """body{font-family:system-ui,sans-serif;max-width:640px;
margin:48px auto;padding:0 16px;color:#222}h1{font-size:24px}
.box{border:1px solid #ccc;border-radius:8px;padding:16px 20px;
background:#fafafa}a.btn{display:inline-block;background:#c74634;
color:#fff;padding:10px 18px;border-radius:6px;text-decoration:none}
td{padding:4px 12px 4px 0;vertical-align:top}

.muted{color:#666;font-size:14px}"""


def ask(name, prompt, secret=False):
    value = os.environ.get(name)
    if not value:
        value = getpass.getpass(prompt) if secret else input(prompt)
    return value.strip()


def http_json(url, data=None, headers=None):
    req = urllib.request.Request(url, data=data, headers=headers or {})
    with urllib.request.urlopen(req, timeout=15) as resp:
        return json.loads(resp.read().decode())


def b64url(raw):
    return base64.urlsafe_b64encode(raw).rstrip(b"=").decode()


def jwt_claims(token):
    part = token.split(".")[1]
```

Copyright © 2026, Oracle and/or its affiliates.

```
    padded = part + "=" * (-len(part) % 4)
    return json.loads(base64.urlsafe_b64decode(padded))


def page(title, body):
    return (f"<!doctype html><html><head><meta charset='utf-8'>"
            f"<title>{title}</title><style>{STYLE}</style></head>"
            f"<body><h1>Summit Retail Portal</h1>{body}"
            "<p class='muted'>Test portal for the Zero Trust Security "
            "with OCI workshop.</p></body></html>")


class Portal(BaseHTTPRequestHandler):
    def send(self, status, body, headers=None):
        data = body.encode()
        self.send_response(status)
        self.send_header("Content-Type", "text/html; charset=utf-8")
        self.send_header("Cache-Control", "no-store")

        for key, value in (headers or {}).items():
            self.send_header(key, value)
        self.send_header("Content-Length", str(len(data)))
        self.end_headers()
        self.wfile.write(data)

    def redirect(self, url, headers=None):
        self.send(302, "", {"Location": url, **(headers or {})})

    def current_user(self):
        for part in self.headers.get("Cookie", "").split(";"):
            key, _, value = part.strip().partition("=")
            if key == COOKIE and value in SESSIONS:
                return value, SESSIONS[value]
        return None, None

    def log_message(self, fmt, *args):
        pass  # print only our own sign-in lines

    def do_GET(self):
        url = urllib.parse.urlsplit(self.path)
        query = dict(urllib.parse.parse_qsl(url.query))
        if url.path == "/":
            return self.home()
        if url.path == "/login":
            return self.login()
        if url.path == "/callback":
            return self.callback(query)
        if url.path == "/logout":
            sid, _ = self.current_user()
            SESSIONS.pop(sid, None)
            expire = f"{COOKIE}=; Max-Age=0; {FLAGS}"
            return self.redirect("/", {"Set-Cookie": expire})
        self.send(404, page("Not found", "<p>Not found.</p>"))

    def home(self):
        _, user = self.current_user()
        if not user:
            return self.send(200, page("Sign in", (
                "<div class='box'><p>You are not signed in.</p>"

                "<p><a class='btn' href='/login'>Continue with Oracle SSO"
                "</a></p></div>")))
        rows = "".join(f"<tr><td>{html.escape(k)}</td>"
                       f"<td>{html.escape(str(v))}</td></tr>"
                       for k, v in user.items())
        name = html.escape(user["Name"])
        self.send(200, page("Signed in", (
            f"<div class='box'><p><b>Welcome, {name}.</b> You are signed "
            f"in through Oracle SSO.</p><table>{rows}</table></div>"
            "<p><a href='/logout'>Sign out of this portal</a></p>")))
```

Copyright © 2026, Oracle and/or its affiliates.

```
    def login(self):
        state, nonce, verifier = (secrets.token_urlsafe(32) for _ in "abc")
        PENDING[state] = {"nonce": nonce, "verifier": verifier}
        challenge = b64url(hashlib.sha256(verifier.encode()).digest())
        params = {"response_type": "code", "client_id": CLIENT_ID,
                  "redirect_uri": REDIRECT_URI, "scope": "openid profile",
                  "state": state, "nonce": nonce,
                  "code_challenge": challenge,
                  "code_challenge_method": "S256"}
        query = urllib.parse.urlencode(params)
        self.redirect(META["authorization_endpoint"] + "?" + query)

    def callback(self, query):
        pending = PENDING.pop(query.get("state", ""), None)
        if "error" in query:
            print(f"Sign-in refused by the domain: {query['error']}")
            error = html.escape(query["error"])
            detail = html.escape(query.get("error_description", ""))
            return self.send(403, page("Access refused", (
                "<div class='box'><p><b>Access refused.</b></p>"
                f"<p>The identity domain returned: <code>{error}</code>"
                f"</p><p>{detail}</p></div><p><a href='/'>Back</a></p>")))
        if not pending or not query.get("code"):
            return self.send(400, page("Sign-in failed", (
                "<p>This sign-in link is stale or incomplete. <a href='/'>"
                "Start again</a> in a new private window.</p>")))
        try:
            user = self.finish_sign_in(query["code"], pending)
        except (urllib.error.URLError, KeyError, ValueError) as err:

            print(f"Sign-in failed: {err}")
            return self.send(502, page("Sign-in failed", (
                f"<p>Sign-in failed: {html.escape(str(err))}</p>"
                "<p><a href='/'>Back</a></p>")))
        sid = secrets.token_urlsafe(32)
        SESSIONS[sid] = user
        print(f"Signed in: {user['Username'] or user['Name']}")
        cookie = f"{COOKIE}={sid}; {FLAGS}"
        self.redirect("/", {"Set-Cookie": cookie})

    def finish_sign_in(self, code, pending):
        form = urllib.parse.urlencode({
            "grant_type": "authorization_code", "code": code,
            "redirect_uri": REDIRECT_URI,
            "code_verifier": pending["verifier"]}).encode()
        pair = f"{CLIENT_ID}:{CLIENT_SECRET}".encode()
        headers = {"Authorization": "Basic " + base64.b64encode(pair).decode(),
                   "Content-Type": "application/x-www-form-urlencoded"}
        # The code is exchanged directly with the domain over TLS and the
        # client secret, so the ID token is used without a signature check.
        token = http_json(META["token_endpoint"], form, headers)
        claims = jwt_claims(token["id_token"])
        aud = claims.get("aud")
        aud_ok = aud == CLIENT_ID or (isinstance(aud, list)
                                      and CLIENT_ID in aud)
        if claims.get("nonce") != pending["nonce"] or not aud_ok:
            raise ValueError("The ID token does not belong to this sign-in.")
        bearer = {"Authorization": "Bearer " + token["access_token"]}
        info = http_json(META["userinfo_endpoint"], headers=bearer)
        if info.get("sub") != claims.get("sub"):
            raise ValueError("The user information does not match.")
        username = info.get("preferred_username") or ""
        return {"Name": info.get("name") or username or claims["sub"],
                "Username": username,
                "Identity domain": DOMAIN_URL,
                "Account managed by": "Oracle SSO (no portal password)"}


if __name__ == "__main__":
    print("Summit Retail test portal\n")
```

Copyright © 2026, Oracle and/or its affiliates.

```
DOMAIN_URL = ask("PORTAL_DOMAIN_URL",
                 "Identity domain URL (https://idcs-...): ").rstrip("/")
if DOMAIN_URL.endswith(":443"):
    DOMAIN_URL = DOMAIN_URL[:-4]
CLIENT_ID = ask("PORTAL_CLIENT_ID", "Client ID: ")
CLIENT_SECRET = ask("PORTAL_CLIENT_SECRET", "Client secret (hidden): ",
                    secret=True)
secure = DOMAIN_URL.startswith(("https://", "http://localhost"))
if not secure or not CLIENT_ID or not CLIENT_SECRET:
    sys.exit("Enter an https:// domain URL, a client ID, and a secret.")
try:
    META = http_json(DOMAIN_URL + "/.well-known/openid-configuration")
except urllib.error.URLError as err:
    sys.exit(f"Could not read the identity domain settings: {err}")
print(f"\nPortal running at {BASE}")
print(f"Redirect URL: {REDIRECT_URI}")
print("Open it in a private browser window. Press Ctrl+C to stop.\n")
server = HTTPServer(("127.0.0.1", PORT), Portal)
if TLS:
    context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    context.load_cert_chain(CERT, KEY)
    server.socket = context.wrap_socket(server.socket, server_side=True)
try:
    server.serve_forever()
except KeyboardInterrupt:
    print("\nPortal stopped.")
```

Copyright © 2026, Oracle and/or its affiliates.